

AI for Everyone

From rules to deep learning

Onur Önder

AI for Everyone

Onur Önder

© 2026 Onur Önder. All rights reserved. Marketing and sales rights belong to Fittechs Yazılım Anonim Şirketi. No part of this book may be reproduced, published or transmitted in any form without the written permission of the author.

Published by the author.

Print edition ISBN: [ISBN]

Interactive digital edition ISBN: 978-625-00-5299-0

First edition: [month, year]

Cover and interior design: Onur Önder

Sales and contact: Fittechs Yazılım Anonim Şirketi, Gayrettepe Mah. Yıldız Posta Cad. No: 8/34, İstanbul, Türkiye. E-mail: support@fittechs.com. Web: onuronder.com

The interactive digital edition of this book, with all 45 live demos, is at book.onuronder.com.

Acknowledgments

This book was born on a screen first. Without the people who read the digital edition early, who poked at the demos and who said honestly where they got stuck, these pages would have stayed far more abstract. My thanks go to my first readers, [names] above all.

I owe the text to the eyes that read it line by line and were not shy to say “this part is not clear”: [names]. Every error that remains is mine.

The ideas this book rests on are not mine. From Turing to Hinton, from the people who wrote rules to the people who computed probabilities, I am indebted to the researchers who wrote this story and to everyone who shared what they learned in the open. The bibliography names only a few of them.

Finally, to my family, who put up with someone saying “one more chapter” for months: [names]. This book is yours too.

Onur Önder

Preface

This book begins with a question: Can a machine think? The question is not new. Long before computers, four hundred years ago, it kept philosophers busy. What is new is that the machines that can try to answer it are now sitting on the table.

I am going to tell a two-century story, from the first calculating gears to today's chatbots. Its heroes are ideas: rules, search, probability, learning, neurons, attention. Each one was born where the one before it hit a wall. That is why the chapters follow the order of history; each chapter shows why the previous idea was not enough and calls the next one onto the stage.

In the digital edition of this book you tried every idea with your own hands: you ran a Turing machine step by step, turned the inputs of a neuron, pulled a picture out of noise. We cannot do that on paper. Instead, I have carried out every experiment for you on the page: step by step, with numbers, with drawings. Then I hand the turn to you. The “Try it yourself” exercises are solved with pencil and paper, and the link to the live demo sits under every figure.

I wrote the book at two depths. The main text is for someone with no background at all. The “Technical depth” boxes are for anyone curious about the formula, the term and the math. You can read the book while skipping the boxes; nothing will be missing. When you get curious, you go back.

I have tried to explain what everyone who talks about artificial intelligence needs to know, without leaving anyone out. I hope that when you finish, you will have given your own answer to the question we started with.

Onur Önder

[place, date]

How to Read This Book

The book has eight chapters. Each chapter covers one era and the main idea of that era, and the chapters were written to be read in order. Still, every chapter stands on its own; you can also start wherever your curiosity takes you.

Two depths. The main text is written in plain language and is complete on its own. At the end of some sections there are boxes headed **Technical depth**. They retell the same idea with its formula, its terms and its math. The boxes can be skipped; the main text does not depend on them. This is the paper version of the dial on the cover: one idea, two depths.

Margin notes. The short notes beside the text give the link between a topic and today, or a detail that is easy to miss at first glance.

Figures. Each of the 45 live demos in the digital edition has become a figure block here. Every block follows the same order:

- *Setup*: what you see in the figure.
- *Step by step*: the experiment, carried out on paper with real numbers.
- *What is happening?*: a one-paragraph explanation.
- *Try it yourself*: one or two questions to solve with pencil and paper. Answers are at the back of the book.

Figures that show a process work like a film strip: follow the frames from left to right, row by row.

Live demos. The QR code under each figure opens the live version of that experiment on your phone. No sign-in or purchase is needed; only that demo opens. The whole book, with all of its demos, is in the digital edition. A list of all demos and their links is at the back of the book.

Test yourself. Every chapter ends with a short quiz. The answer key is at the back of the book. Some figures are tests as well (“Self-

test”); their answers and the reasoning are in the same place.

What to keep from this chapter. The bullet list at the end of each chapter sums up what should stay with you from that chapter. Once you have finished the book, reading only these lists is a good way to review.

Contents

Chapter 1 · Minds and Machines	11
1.1 Can a machine think?	12
1.2 What is intelligence?	13
1.3 Is thinking = computing?	16
1.4 From Babbage to Turing	20
1.5 ENIAC and the birth of the modern computer	24
1.6 Narrow AI or General AI?	27
1.7 Why now? Moore’s law	29
1.8 Test yourself	33
Answer Key	141
Glossary	147
Bibliography and Further Reading	157
Live Demos	161
Index	165

1

CHAPTER 1

Minds and Machines

1.1 Can a machine think?

Once upon a time... except this story isn't over, and you are in it. It began long before computers, with a question: What is thinking? If some kind of gear turns inside our heads, couldn't a machine turn the same gear? People have argued about this for centuries.

This first chapter goes back to the very start of the road. What is intelligence? Can “thinking” really be written down as a step-by-step recipe? We'll walk from Babbage's gears to the birth of the modern computer. And at the end of the road, it will become clear why artificial intelligence (AI for short) is possible at all.

Margin note. AI isn't really new: “can machines think?” has occupied philosophers since the 1600s. What's new is the hardware that can test it.

TECHNICAL DEPTH

AI was born at the intersection of computability theory and the philosophy of mind. The core claim: if mental processes can be described by formal rules, then in principle a machine can carry them out too.

This chapter starts from defining intelligence, lays the foundation stones of computation (binary representation, algorithms, the Turing machine, the stored-program architecture) and connects to the modern debate via the weak/strong AI distinction. That gives the later learning and neural-network ideas solid ground to stand on.

Let's start the road at the first question. That “intelligence” we want to hand to a machine: what is it, exactly?

1.2 What is intelligence?

It looks like an easy question, but “what is intelligence?” has no single answer. Is the puzzle-solver smart, or the person who instantly finds their way in a new city, or the friend who reads your sadness in one glance? Perhaps all of them; yet each is a very different skill.

Psychologist Howard Gardner said exactly this: Intelligence doesn’t fit in one number (IQ); it comes in several kinds. Explore them below, and notice the interesting part: today’s AI is a master of some and still a toddler at others.

***Margin note.** A calculator is a million times faster than you at arithmetic but can’t laugh at a joke. Being “smart” isn’t a single axis.*

Figure 1.1 • Explore multiple intelligences

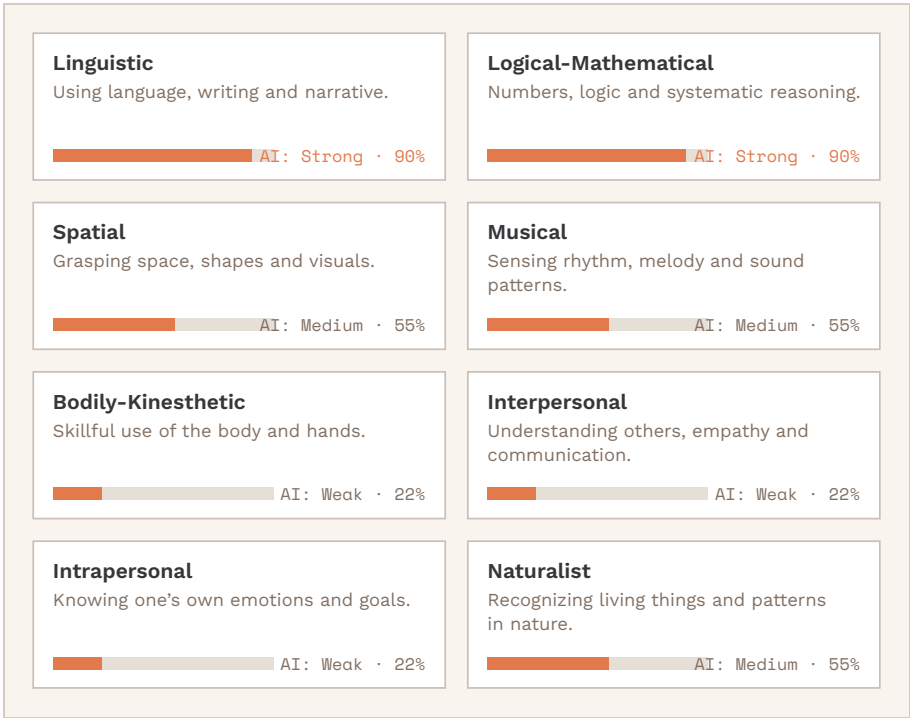


Figure 1.1

Setup. The figure shows eight cards, one for each kind of intelligence on Gardner’s list. The bar under each card shows where today’s AI stands in that area. A full bar means “strong,” a half bar “medium,” a short stub “weak.” On screen the cards opened one at a time; on paper you see all eight side by side.

Step by step. All eight cards, in the source order:

Intelligence type	What it means	AI today	Bar
Linguistic	Using language, writing and narrative.	Strong	90%
Logical-Mathematical	Numbers, logic and systematic reasoning.	Strong	90%
Spatial	Grasping space, shapes and visuals.	Medium	55%

Musical	Sensing rhythm, melody and sound patterns.	Medium	55%
Bodily-Kinesthetic	Skillful use of the body and hands.	Weak	22%
Interpersonal	Understanding others, empathy and communication.	Weak	22%
Intrapersonal	Knowing one's own emotions and goals.	Weak	22%
Naturalist	Recognizing living things and patterns in nature.	Medium	55%

Count the table and you get two “strong,” three “medium,” three “weak.” What do the two strong ones have in common? Both work with symbols: letters, numbers, rules. A computer runs on exactly that kind of material; the binary code in the next section is a string of symbols too.

The three weak ones ask for a different material: a body, the face of the person across from you, your own inner world. None of these turns into symbols easily. The middle three, space, music and nature, sit between the two. Part of sound and image can be turned into numbers; part of it still resists.

Look at the order as well: Gardner's list is not a ranking. The cards only count separate abilities; none of them is “more intelligence” than another. The same holds for AI: being strong in language does not mean being strong in the other seven areas.

What is happening? For Gardner, intelligence isn't one number but eight distinct families of ability; the card you tapped is one of them. Notice something too: today's AI is great with language and logic, yet far behind a small child at anything involving the body or emotions.

Try it yourself. 1) Close the book and list the eight kinds of intelligence from memory. How many did you recall, and which ones slip away? 2) Think about yesterday: from morning to night, which three kinds did you use most? According to the table, in how many of those is today's AI strong? 3) What do the three “weak” kinds in the

table have in common? Write it in one sentence. Live demo:



book.onuronder.com/d/en/40ee8a0d04

TECHNICAL DEPTH

There is no agreed single definition of intelligence; working definitions cluster around “goal-directed adaptation, abstraction, learning and problem-solving capacity.” The AI field uses a pragmatic one: systems that perform tasks requiring human intelligence.

Gardner’s Theory of Multiple Intelligences (1983) splits intelligence into relatively independent domains: linguistic, logical-mathematical, spatial, musical, bodily-kinesthetic, interpersonal, intrapersonal and naturalistic. The theory is contested in psychometrics, but it usefully shows that intelligence is multi-dimensional: today’s AI is wildly uneven across these dimensions.

This is one of the eight domains in Gardner’s theory of multiple intelligences (contested in psychometrics). The real lesson: intelligence is not one-dimensional, and AI is wildly uneven across these dimensions; strong in language and logic, weak in bodily and social/intrapersonal domains.

If intelligence is not one piece, can at least one of its pieces be written down as a step-by-step recipe? The next section looks at that question from the kitchen.

1.3 Is thinking = computing?

Step into the kitchen for a moment. You hold a cake recipe: crack the eggs, whisk, add the flour, into the oven. Someone following that

recipe to the letter will produce a cake without “understanding” baking at all. The idea beneath all of AI is exactly that: Maybe “thinking,” too, is applying small, mechanical steps in order.

If so, we can get a machine to do those steps too. That takes two things:

- A simple alphabet the machine can use (binary: 0 and 1)
- A clear list of steps (the recipe itself: an algorithm)

See how binary works for yourself below.

***Margin note.** The word “algorithm” comes from the name of the 9th-century Persian mathematician al-Khwarizmi. Binary itself was formalized by Leibniz in 1703.*

Figure 1.2 • Crack the binary code

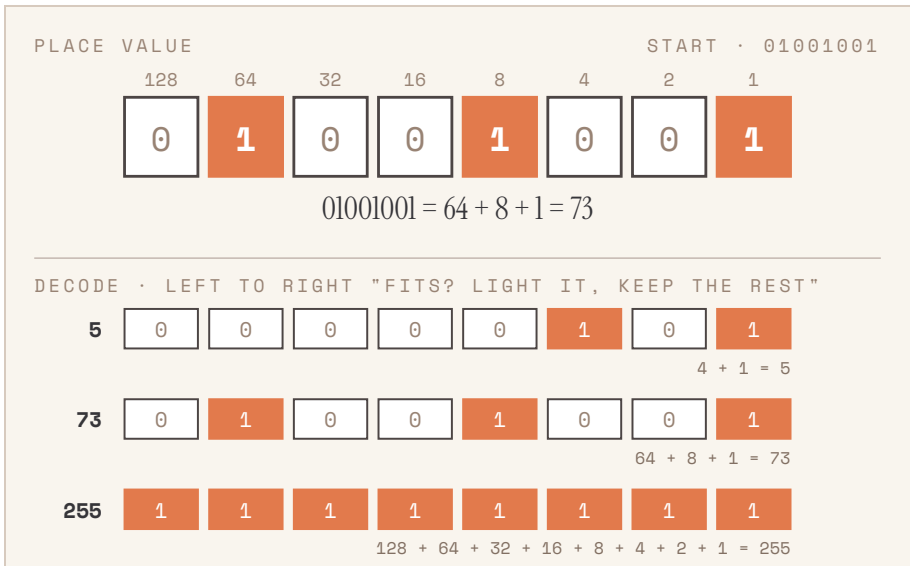


Figure 1.2

Setup. The figure shows eight boxes in a row. Above each box is its place value; from left to right, 128, 64, 32, 16, 8, 4, 2 and 1. A lit box means 1, a dark box means 0. The figure shows the opening state of

the demo: 01001001. The number underneath is the sum of the place values of the lit boxes.

Step by step. First, the place-value chart:

Box	1	2	3	4	5	6	7	8
Place value	128	64	32	16	8	4	2	1
Start	0	1	0	0	1	0	0	1

1. Decode the opening number. The lit boxes are 2, 5 and 8; their values are 64, 8 and 1. Add them: $64 + 8 + 1 = 73$. So 01001001 = 73.
2. Now go the other way and write the number 5 into the boxes. Start from the left and ask each box: “Does your value fit into the number?” 128, 64, 32, 16 and 8 do not fit; they all stay dark. 4 fits: light it, and 1 is left. 2 does not fit. 1 fits: light it, and 0 is left. The result is 00000101.
3. Write 73 the same way. 128 does not fit. 64 fits, 9 left. 32 and 16 do not fit. 8 fits, 1 left. 4 and 2 do not fit. 1 fits, 0 left. The result is 01001001, the same pattern as the opening state.
4. Finally, 255: light every box. $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$, that is, 11111111. This is the largest number eight boxes can say. One more, 256, needs a ninth box.

Did you spot the rule? Going from left to right, “if it fits, light it and carry on with the remainder” is all it takes. Every number has exactly one pattern; two different patterns never give the same number. Eight boxes hold 256 different numbers, from 0 to 255; computer people call that a byte.

What is happening? Eight boxes, eight bulbs: off means 0, on means 1. But who handed out the values? Think of it this way: The rightmost box holds the smallest, a 1. Moving left, each box carries double its right-hand neighbor: 1, 2, 4, 8, 16, 32, 64, and 128 on the far left. Why always double? Because each new box must be able to say one more than all the boxes to its right combined; otherwise some numbers

could never be written. Now add up the values of the lit boxes: there is your number. A computer's whole world is this game of on and off.

Try it yourself. 1) Write the number 10 with eight boxes. 2) Which number is the pattern 10110000? 3) If only the leftmost box is lit, what is the number? If you added a ninth box, what would the largest number be? Live demo:



book.onuronder.com/d/en/4642d3916a

TECHNICAL DEPTH

This view is called computationalism: the mind is an information-processing system, and cognition is computation in the form of symbol manipulation. Its roots reach back to Hobbes (“reasoning is reckoning”); its modern form developed with Putnam and Fodor.

In practice, two building blocks are needed. Binary representation: all information is encoded in base-2 (bits); Boolean algebra (Boole, 1854) and Shannon’s 1937 thesis showed that logic can be realized with electrical switches. Algorithm: a finite sequence of well-defined steps (the term comes from the 9th-century mathematician al-Khwarizmi). Together they make “thinking” mechanically executable.

Binary 01001001 = the sum of its place values = $64 + 8 + 1 = 73$. Each bit represents 128, 64, 32, ... from left to right. 8 bits (1 byte) encode any number from 0 to 255.

General rule: value = $\sum b_i \cdot 2^i$, with $i = 0 \dots 7$ counted from right to left; b_i is the 0 or 1 of that bit. n bits encode 2^n distinct numbers; for 8 bits, $2^8 = 256$.

The alphabet and the recipe are ready. But who will carry out the recipe, and on which machine? The answer begins in the 1800s, with a dream of gears.

1.4 From Babbage to Turing

The dream of a “computer” was first dreamed in the 1800s. Charles Babbage drew a giant calculating machine of gears and levers: the Analytical Engine. Money and years ran out; he died without ever seeing it built. But his friend Ada Lovelace wrote the first “recipe” for that imaginary machine. That is why she counts as history’s first programmer.

About a century later, Alan Turing carried the dream a step further. Picture a single little box that reads a tape, writes on it and slides left or right. In principle, it can perform any calculation. It is called a Turing machine. Watch a real one “think” frame by frame in Figure 1.3.

Margin note. In 1948 Turing designed a chess algorithm called “Turochamp.” No machine of the era could run it, so Turing worked out the moves by hand on paper.

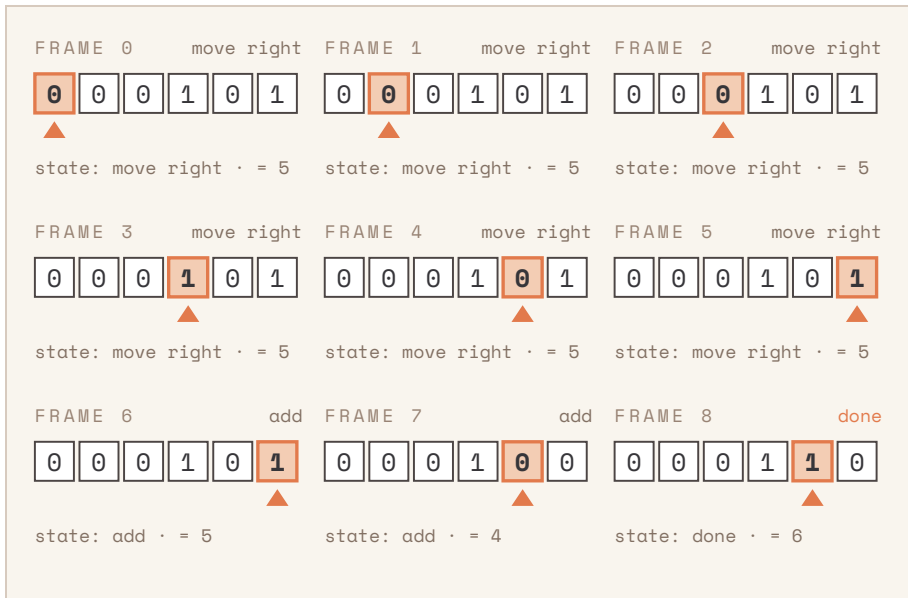
Figure 1.3 · A working Turing machine (+1)

Figure 1.3

Setup. The figure shows a tape of six cells; the arrow (▲) moving above it is the machine's head. The tape reads 000101, which is 5 in binary. The machine has three states: “move right,” “add 1 (carry)” and “done.” The nine frames of the film strip show every step taken while 1 is added to 5.

Step by step. Read the frames in order; each row gives the tape, the head position and the state. The cells are numbered 0 to 5 from left to right.

0. Start. The head is on the leftmost cell (0), the state is “move right.” The tape reads 0 0 0 1 0 1, that is, 5.
1. The head slides one cell to the right (1). The tape does not change. The head (▲) steps right to find the number's rightmost digit. Just like adding by hand, the work starts from the far right.
2. The head is on cell 2. It is still only reading, not writing.
3. The head is on cell 3; there is a 1 here, but in this state the machine ignores it and moves on.
4. The head is on cell 4.

5. The head is on the last cell (5). The tape still reads 0 0 0 1 0 1.
6. No cell is left on the right. The machine changes state: “add 1 (carry).” The head stays put, the tape stays the same.
7. The head sees a 1 on the last cell. It turns the 1 into 0 and carries one to the left; the head moves to cell 4. The tape reads 0 0 0 1 0 0, for a moment 4. The head now works backward, adding 1 leftward: every 1 it sees becomes 0 with a carry, and at the first 0 it writes 1 and stops. The same “carry the one” rule you learned in school.
8. The head sees a 0 on cell 4. It writes a 1 there and stops; the state is “done.” The tape reads 0 0 0 1 1 0, that is, 6. Done. With only a handful of simple rules like “read, write, move left/right,” the machine added 1 to a binary number. This is exactly what it means for a Turing machine to compute.

Eight moves in all: five steps to the right, one state change, one clear, one write. Five plus one made six; the tape says it as 000110.

What is happening? The machine walks the tape like an ant: first it heads right to find the end of the number, then turns back and adds 1. The way it carries the digit is no different from how you carry in addition on paper. Such tiny moves can carry out any calculation; that is the whole magic.

Try it yourself. 1) Start the tape at 001111 (15) and draw the frames yourself on paper. How many 1s get cleared, in how many moves does the machine finish, and which number is left on the tape? 2) Start the tape at 010110 (22). When the head reaches the end and turns back, what is the first digit it sees? In how many moves does it finish? 3) What would happen with a tape of 111111 (63)? The machine sees a 1 even on the leftmost cell; it cannot move left, so it stops.

Live demo:



book.onuronder.com/d/en/6c232ac487

TECHNICAL DEPTH

Babbage’s Analytical Engine (1837) was a general-purpose design with a “mill” (processing unit) and a “store” (memory), programmable via punched cards. Ada Lovelace’s 1843 notes contain the algorithm for computing Bernoulli numbers, generally regarded as the first computer program.

Alan Turing’s 1936 paper “On Computable Numbers” introduced the abstract Turing machine and the universal Turing machine; this is the formal basis of computability. The simulation below is a small Turing machine that adds 1 to a binary number, doing arithmetic with only read/write and state transitions.

The machine slides right to the end of the tape, then turns left and adds 1 to the binary number (carry logic): the 1s it sees become 0, and the first 0 becomes 1. It performs addition using only read/write and state transitions.

The transition table of the machine in Figure 1.3, taken from the source code:

State	Read	Write	Move	New state
move right	0 or 1	same	right	move right; at the end of the tape, “add 1”
add 1	1	0	left	add 1; at the start of the tape, “done”
add 1	0	1	stay	done

Three rows, a six-cell tape and a single head: enough to add 1 to any number from 0 to 62. For 63 the tape overflows and 0 remains.

Turing’s machine was a thought on paper. Pouring it into real tubes and wires is another story.

1.5 ENIAC and the birth of the modern computer

In 1945 a giant machine roared to life: ENIAC. It weighed tons and ran on tens of thousands of tubes. And this first electronic computer had a grumpy streak: teaching it a new job meant days of unplugging and replugging cables by hand.

The fix was an elegant idea: Keep the recipe next to the ingredients, in memory. Then, instead of rewiring the machine, you simply load new “instructions.” Nearly every computer today, including the phone in your pocket, runs on this arrangement. Follow the cycle in the figure below, then look at what its parts do.

Margin note. Reprogramming ENIAC took days; the cables had to be reconnected by hand. That is why “putting the program in memory” was a revolution.

Figure 1.4 • The Fetch – Execute – Write cycle

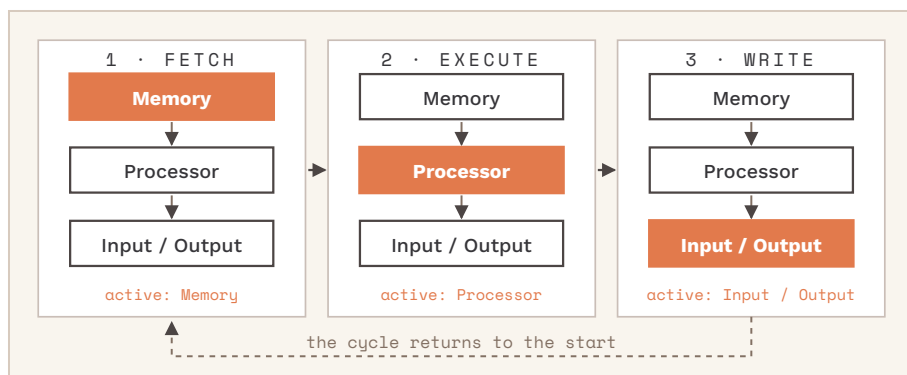


Figure 1.4

Setup. The figure shows three boxes: Memory, Processor and Input / Output. On screen these boxes lit up and dimmed in turn. On paper you see three frames side by side: “1 · Fetch,” “2 · Execute,” “3 · Write.” In each frame the part at work in that phase is dark. After the third frame the cycle starts over.

Step by step. The parts first; the descriptions come from the cards in the demo:

Phase	Active part	What it does
1 · Fetch	Memory	Stores both the program (instructions) and the data together. The real revolution is here: instructions are kept just like data.
2 · Execute	Processor	The ALU computes (adding, comparing); the control unit fetches instructions from memory in order and executes them.
3 · Write	Input / Output	The link to the outside world: keyboard, screen, sensors. Data comes in here, results go out here.

Now turn the cycle with a small program. Let memory hold two lines: the first line says “add 5 and 3,” the second says “write the result to the screen.” The numbers 5 and 3 sit in memory too.

1. Fetch. The control unit reaches into memory and takes the instruction on the first line: “add 5 and 3.” The instruction came from memory just like 5 and 3 did; it is data, not a cable.
2. Execute. The ALU does the addition: $5 + 3 = 8$. For now the result stays inside the processor.
3. Write. The 8 is put back into memory. Nothing appears on the screen in this round.
4. Fetch. The cycle has come back to the start; the control unit takes the second line: “write the result to the screen.”
5. Execute. The processor reads the 8 from memory and prepares it for output.
6. Write. An 8 appears on the screen. The two-line program is finished; if there is a next instruction, the cycle goes on.

Two rounds, six phases. On ENIAC, changing the first line meant days of pulling cables. Here the only thing to do is write a different instruction into the first line of memory, say “multiply 5 by 3.” The cycle stays the same; only the instruction it picks up changes. The whole power of the stored program is in that sentence.

What is happening? A computer performs a three-step dance non-stop: grab the next instruction from memory (fetch), do what it says (execute), save or display the result (write). This is the loop beating at the heart of your phone and your computer alike.

Try it yourself. 1) You typed 7×6 into the calculator on your phone. Describe the three phases in your own words; which part is at work in each one? 2) Add a third line to the program above: “multiply the result by 2.” How many more phases are needed, and which number is written to memory? 3) Picture a processor that runs 3 billion cycles per second. Each cycle has three phases. How many phases in one second? Live demo:



book.onuronder.com/d/en/c4fdfa18f9

TECHNICAL DEPTH

ENIAC (1945; Mauchly & Eckert) was a general-purpose electronic digital computer; it used ~17,500 vacuum tubes and was programmed by rewiring plugboards (its first programmers were six women mathematicians). Internally it worked in decimal, not binary.

John von Neumann’s 1945 EDVAC report popularized the stored-program principle: program and data live in the same memory. The architecture consists of a processor (ALU + control unit), memory and input/output, talking over a bus. The processor runs a continual “fetch–decode–execute” cycle. The “von Neumann bottleneck” is this design’s well-known limit.

In the “fetch” phase, the control unit reads the next instruction from memory; the program sits in memory just like data (the stored-program principle). In “execute” the processor does the work; in “write” the result goes out.

The machine can now run any recipe. But when we call it “artificial intelligence” today, how much of that do we actually mean?

1.6 Narrow AI or General AI?

The all-understanding AI of the movies doesn’t exist yet. Every AI today is “narrow”: a champion at its own job, a beginner one step outside it. A chess engine can beat you, yet you can’t ask it for a soup recipe.

The hypothetical system that can learn across every domain like a human is called “artificial general intelligence” (AGI); nobody has built one yet. Sort the examples below: Which exists today, and which is still science fiction?

Margin note. A system doing many tasks well (general) and a system truly “understanding / being conscious” (strong) are different questions. Keeping them apart is key to the modern AI debate.

Figure 1.5 • Real today, or science fiction?

#	Example	Narrow AI • exists today	General / AGI • not yet
1	Chess engine	☐	☐
2	Face recognition system	☐	☐
3	Chatbot (language model)	☐	☐
4	A machine that learns any profession like a human and has its own goals	☐	☐
5	A self-aware, conscious AI	☐	☐

Figure 1.5

Setup. The figure shows two columns. The left one is headed “Narrow AI • exists today,” the right one “General / AGI • not yet.” Five cards sit in the middle. On screen each card was placed in a column,

and a right choice lit up green, a wrong one red. On paper the boxes next to the cards are empty; you fill them in with a pencil. There is no score, and there is no third column.

Self-test. Put each card in a column. There is only one test: Is the system a master of one job, or can it learn a job it has never seen, the way a human does? While you decide, look not at how impressive the card is but at how many jobs it knows. Answers and reasons are at the back of the book.

#	Example	Narrow AI exists today	General / AGI not yet
1	Chess engine	☐	☐
2	Face recognition system	☐	☐
3	Chatbot (language model)	☐	☐
4	A machine that learns any profession like a human and has its own goals	☐	☐
5	A self-aware, conscious AI	☐	☐

After you have marked all five, look again: How many cards are in the left column, how many in the right? What do the cards on the right have in common? If you can write that in one sentence, you have caught the main idea of the chapter.

What is happening? Here is the trick: any system that masters just one job is “narrow,” even chatbots. “General” means a machine that can learn across every domain like a human, and no such machine exists yet. A conscious machine is a different story altogether.

Try it yourself. 1) Write down three AI examples from your own daily life: navigation, translation, movie recommendations. Give each one a column. 2) The fourth and fifth cards are in the same column; are they the same thing? Apply the two questions in the margin note (“general” and “strong”) to each of them separately. 3) What happens if you ask a chess engine “give me a soup recipe”? Give your answer

by writing the definition of narrow AI in one sentence. Live demo:



book.onuronder.com/d/en/e69cfc21ea

TECHNICAL DEPTH

Weak (narrow) AI refers to systems that perform specific tasks without genuine understanding or consciousness; all of today's systems fall here. The terms were coined by philosopher John Searle (1980, the Chinese Room argument).

An important distinction: Searle's "strong AI" is really about whether the machine truly understands (has a mind); in everyday usage it is often conflated with "artificial general intelligence" (AGI). AGI is a hypothetical capability level that generalizes across domains; "strong AI" is a philosophical claim. They are not the same thing.

Careful: "general" (AGI, cross-domain generalization) and "strong AI" (Searle: does the machine truly understand / is it conscious?) are different questions. Even AGI carries no consciousness requirement: being general does not mean being conscious.

The ideas had been on the table for a hundred years. So why did even narrow AI enter our lives only in the last few years?

1.7 Why now? Moore's law

The ideas were ready by the mid-1900s; the machines were feeble. Do you know the old rice fable? One grain on the first square of a chessboard, double on every square after... halfway across the board,

no granary on earth is enough. In 1965 Gordon Moore noticed the transistors on chips doubling roughly every two years, just like that.

Doubling looks innocent, but repeated it explodes. Try it yourself below: in the table of Figure 1.6, double the number every 2 years and watch it rocket. The power carrying modern AI is exactly this compounding computation.

Margin note. If you could fold a sheet of paper 42 times, it would reach the Moon. That is the power of exponential growth; chips grew at that pace for decades.

Figure 1.6 • Feel exponential growth

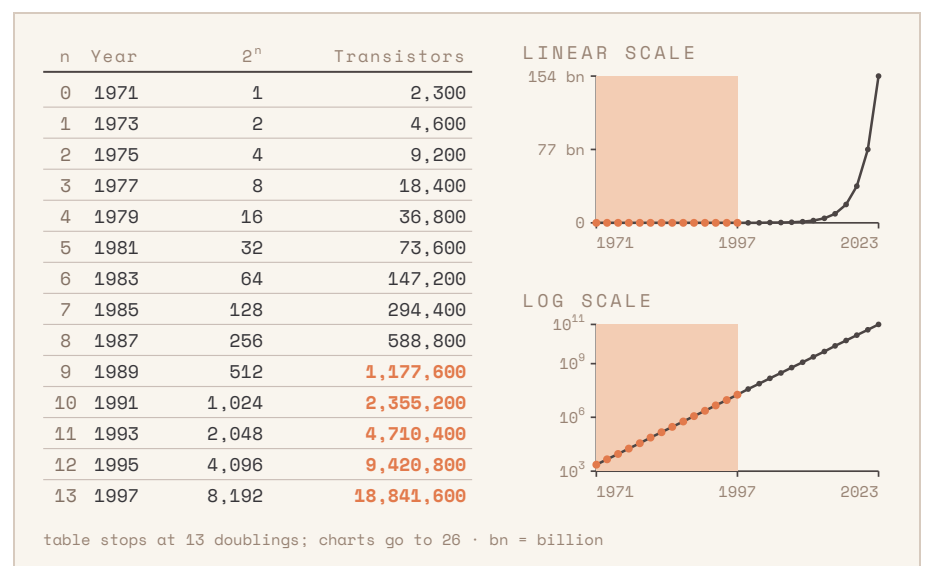


Figure 1.6

Setup. The figure starts in 1971. That year the first microprocessor, the Intel 4004, went on sale with 2,300 transistors. Each step in the figure means “two years passed, the count doubled.” On screen this was a counter; with every doubling the year rose by two and the number grew. On paper the same thing is a table. The counter stops after 26 doublings, in 2023.

Step by step. Follow the table row by row. The third column is 2^n , the fourth is $2,300 \times 2^n$. Each row is exactly twice the one before; there is no other rule.

Doubling (n)	Year	2^n	Transistors
0	1971	1	2,300
1	1973	2	4,600
2	1975	4	9,200
3	1977	8	18,400
4	1979	16	36,800
5	1981	32	73,600
6	1983	64	147,200
7	1985	128	294,400
8	1987	256	588,800
9	1989	512	1,177,600
10	1991	1,024	2,355,200
11	1993	2,048	4,710,400
12	1995	4,096	9,420,800
13	1997	8,192	18,841,600

The first rows behave: 2,300, 4,600, 9,200. In ten years the count grows 32-fold but is still in the tens of thousands. In 1989 it crosses the one-million line: 1,177,600, or “1.2 million” as the demo puts it. In 1997 it is 18.8 million. The table stops after 13 doublings, 26 years; the demo goes on to 26 doublings.

What if you keep going? After 20 doublings, in 2011, $2^n = 1,048,576$ and the count is 2,411,724,800; the demo shows it as “2.4 billion.” After 26 doublings, in 2023, $2^n = 67,108,864$ and the count is 154,350,387,200: “154.4 billion.” Between the first row and the last there is a factor of 67 million. In the rice fable the granaries ran out on the 32nd square; chips reached the 26th square in 2023.

What is happening? Every press means “two years passed, the power doubled.” Within a few presses the number takes off; that is what exponential growth is like. Computers grew at this pace for decades, and that accumulation is what made today’s AI possible.

Try it yourself. 1) Extend the table by two rows: 2^n and the transistor count for 1999 and 2001. 2) Starting from 2,300, which row is the first to pass one million; how many years did it take? 3) If the doubling time were 3 years instead of 2, how many transistors would there be in 1995? Compare with the 1995 value in the table. Live

demo:



book.onuronder.com/d/en/eb5ac74b07

TECHNICAL DEPTH

Moore's law is not a law of nature but an empirical observation and economic trend: the transistor count on an integrated circuit roughly doubles every two years (Moore's 1965 observation, revised in 1975). This exponential growth made the heavy matrix computation deep learning needs economical.

The exponential trend held for ~50 years (1971 Intel 4004: ~2,300 transistors → 2020s: tens of billions). It has been slowing recently due to physical limits (subatomic scale, heat); the industry now leans on parallelism and specialized AI hardware such as GPUs/TPUs.

Exponential growth: $\times 2$ roughly every 2 years; n doublings give a 2^n -fold increase. It is not a law of nature but an empirical trend, and it has been slowing in recent years due to physical limits.

The formula behind Figure 1.6: $N(t) = N_0 \cdot 2^{(t/2)}$, where t is the number of years since 1971 and $N_0 = 2,300$. For $t = 52$, $N = 2,300 \cdot 2^{26} \approx 1.5 \times 10^{11}$. Real chips sit a little below this curve; the doubling time has stretched since the 2000s.

The building blocks of the chapter are in place: intelligence, binary code, algorithm, Turing machine, stored program, the narrow-versus-general distinction, exponential growth. See how much of it stayed with you in the questions below.

1.8 Test yourself

Answers are at the back of the book.

1. What did Howard Gardner propose?
 - a) That IQ fully measures intelligence
 - b) That the brain is a computer

- c) That machines can never think
 - d) That intelligence has many kinds, not one
2. Binary code is made of which symbols?
- a) 0 and 1
 - b) A to Z
 - c) Dots and dashes
 - d) 0–9
3. What can a Turing machine fundamentally do?
- a) In principle any computation, via simple rules
 - b) Only addition
 - c) Only play chess
 - d) Only store text
4. The core idea of the von Neumann architecture?
- a) Inventing the internet
 - b) Keeping program and data in the same memory
 - c) Using binary instead of decimal
 - d) Rewiring for each new task
5. What kind are today's AI systems?
- a) General AI (AGI)
 - b) Conscious AI
 - c) Narrow AI
 - d) Strong AI
6. What does Moore's law say?
- a) Internet speed is constant
 - b) AI will surpass humans
 - c) Computers get cheaper yearly
 - d) Transistor count ~doubles every 2 years

What to keep from this chapter

- Intelligence is not one number; across Gardner's eight kinds, today's AI is strong in language and logic, weak in body and emotion.